

Extension of a RISC-V Core with Custom Matrix Multiplication instructions

Erklärung zur Abschlussarbeit

Hiermit versichere ich, dass die eingereichte Ausarbeitung von mir persönlich verfasst und meine eigene individuelle Prüfungsleistung ist.

Ich versichere, dass die Ausarbeitung und auch keine Teile davon durch künstliche Intelligenz (KI) dergestalt erstellt wurden, dass das KI-Werk bzw. KI-Werkteile meine eigene Prüfungsleistung ersetzen. Ich versichere, KI allenfalls eingesetzt zu haben, um einen von KI für meine Aufgabenstellung ausgearbeiteten Lösungsvorschlag kritisch zu beurteilen und/oder einen Überblick über Aspekte zu erhalten, die für die von mir in Eigenleistung zu erbringende Prüfungsleistung relevant sein könnten. Soweit durch die Aufgabenstellung bzw. Hinweise der Prüfenden der Einsatz von KI vorgegeben ist, sind die von KI erzeugten Werkteile von mir in der Arbeit entsprechend gekennzeichnet. Mir ist bewusst, dass die von KI erzeugten Werkteile auf ihre Validität zu überprüfen und nicht zitierfähig sind.

Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden.

Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Literatur und sonstige Quellen sind nachgewiesen und im Literatur- und Quellenverzeichnis aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann.

Ort, Datum

Unterschrift des Verfassers / der Verfasserin

Veröffentlichung der Arbeit in der Bibliothek der Technischen Hochschule Aschaffenburg

Der Veröffentlichung der Bachelorarbeit in der Bibliothek der Technischen Hochschule Aschaffenburg wird **[nicht]** zugestimmt.

Ort, Datum

Unterschrift des Verfassers / der Verfasserin

Extension of a RISC-V Core with Custom Matrix Multiplication instructions

Bachelorarbeit von
Wolfgang Bischoff

13. Januar 2026

Extension of a RISC-V Core with Custom Matrix Multiplication instructions

Extension of a RISC-V Core with Custom Matrix Multiplication instructions

Autor:
Wolfgang Bischoff
2228538

Erstprüfer:
Prof. Dr. Christian Jakob HDA



TECHNISCHE HOCHSCHULE ASCHAFFENBURG

FAKULTÄT INGENIEURSWISSENSCHAFTEN

WÜRZBURGER STRASSE 45

D-63743 ASCHAFFENBURG

Vorwort

Ich bedanke mich bei Herrn Prof. Dr. Christian Jakob für die Betreueung der Arbeit auf einem Themengebiet, dass mich persönlich sehr interessiert und viel Freude bereitet. Ich bedanke mich bei der TH Aschaffenburg und allen beteiligten Mitarbeiterinnen und Professoren für die Organisation eines berufsgeleitenden Studiums.

„Wenn du es nicht versuchst, wirst du nie wissen, ob du es kannst.“

Hans Kammerlander

Inhaltsverzeichnis

Abbildungsverzeichnis	VIII
Tabellenverzeichnis	X
Glossar	X
Formelverzeichnis	XI
1 Einleitung	1
1.1 Motivation	1
1.2 Aufgabenstellung	1
1.3 Struktur der Arbeit	2
2 Stand der Technik	3
3 Theoretische Grundlagen	6
3.1 Vektoren und Matrizen	6
3.2 Matrixmultiplikation mit SISD	7
3.3 Matrixmultiplikation mit Strip-Mining und SIMD	8
3.4 Die RISC-V Vector Extension	9
3.5 Blockweiße Matrix Multiplikation	14
3.6 Outer-Produkt	15
4 Durchführung und Methodik	17
4.1 Der NEORV32 Core und dessen Einsatz	17
4.2 Architektur des NEORV32 Core	18
4.3 Die Fetch-Engine	19
4.4 CPU-Control und Execution-Engine	19
4.5 Die Load-Store-Engine	21
5 Erweiterung des NEORV32-CPU	22
5.1 Die Matrix-Register	22
5.2 Die Instruktionen	22
5.3 Erweiterung der Execution-Engine	24
5.4 Erweiterung der Load-Store-Unit	27
6 Auswertung und Evaluierung	29
7 Zusammenfassung und Ausblick	30
Anhang	31
Quellenverzeichnis	32

Abbildungsverzeichnis

3.1	Ansätze zur Matrix-Extension	13
3.2	Matrix Multiplikation blockweise	14
3.3	Matrix Multiplikation Ergebnis	15
3.4	Matrix Multiplikation Berechnungsvorschrift	15
3.5	Matrix Multiplikation Blöcke A, E, B und G	15
3.6	Matrix Accumulator (MAC)	16
4.1	NEORV32 Architektur	18
4.2	NEORV32 Gültige Instruktion	19

Tabellenverzeichnis

Glossar

RVV

RISC-V Vector Extensions

SISD

Single Instruction Single Data

SIMD

Single Instruction Multiple Data

XLEN

Standardregisterlänge in Bits

VLEN

Vector-Registerlänge in Bits

MLEN

Matrix-Registerlänge in Bits

AVL Application Vector Length

SEW

Selected Element Width

LMUL

Group Modifier

VL Vector Length

IME Integrated Matrix Extension

VME

Vector Matrix Extension

AME

Attached Matrix Extension

MAC

Matrix Accumulator

CSR

Configuration and Status Register

Formelverzeichnis

1 Einleitung

1.1 Motivation

In dieser Arbeit soll ein RISC-V CPU um Anweisung zur Durchführung von Matrix Multiplikationen erweitert werden.

RISC-V ist eine lizenzfreie Beschreibung eines Processors und der Anweisungen, die der Processor verarbeiten können muss. Durch diese Lizenzfreiheit ist die RISC-V Architektur sehr interessant für den kommerziellen Einsatz. Die Firma Qualcomm als eines der weltweit, führendes Unternehmen auf dem Gebiet der Halbleiter und der Telekommunikation hat eine RISC-V Erweiterung namens Xqci für RISC-V und auch zu den Compiler-Framework LLVM beigesteuert [9]. Qualcomm führt auch Käufe von Firmen mit RISC-V Wissen durch. Dies deutet daraufhin, dass Qualcomm den Einsatz von RISC-V in Produkten ernsthaft in Erwägung zieht.

Matrix-Multiplikation ist die Grundlage für perspektivische Projektionen, Rotationen und Bewegung von Objekten in der 3D Computer-Grafik. Dort ist es notwendig 4x4 Matrizen zu verwenden und mathematischen Berechnungen so schnell wie möglich auf diesen Matrizen ausführen zu können. Der Einsatz von 3D-Computergrafik ist seit mehreren Jahren nicht mehr aus dem kulturellen Alltag aber auch jeglicher Art von Bildgebung nicht mehr wegzudenken.

Getrieben durch den Erfolg von Large Language Models kann sich eine Spezifikation für CPUs nicht mehr durchsetzen, wenn Sie keine Anweisungen für den effizienten Umgang mit Vektoren und Matrizen besitzt. In einem späteren Kapitel wird ein Hinweis darauf gegeben, wie auch die Berechnung von Neuronalen Netze hauptsächlich auf die Berechnung von Matrizen zurückgeführt werden kann. Die CPUs der Firmen AMD und Intel besitzen die Erweiterungen AMX (Advanced Matrix Extensions), AVX2 (Advanced Vector Extensions 2) and FMA (Fused Multiply Add).

All diese Punkte zusammengekommen ergeben eine große Motivation für den Einsatz von RISC-V und den Einsatz von Matrix-Multiplikation innerhalb eines RISC-V CPU.

1.2 Aufgabenstellung

Der NEORV32 RISC-V CPU wird um Anweisungen erweitert, die die Matrix-Multiplikation ermöglichen. Dabei orientieren sich die Anweisungen von der

Kleinteiligkeit her an den Anweisungen, wie sie bereits für die RVV-Vektor-Extension ratifiziert worden sind. Im speziellen wird das Konzept des Strip-Mining ermöglicht. Dabei handelt es sich um die Aufteilung des Problems in Teile, die der CPU, aufgrund von begrenzter Ressourcen, schrittweise zugeführt werden, bis die Aufgabe als ganzes gelöst ist.

Die Erweiterung soll mit dem Wissenstand eines Beginners in der Hardwarebeschreibung vorgenommen werden können. Es wird im speziellen eine Erweiterung des bereits existierenden NEORV32 Core (TODO: Link einfügen) vorgenommen. Aus diesem Grund ergibt sich die Einschränkung, dass sich die Erweiterung in das existierende Design NEORV32 Core einfügen muss.

Die Matrizenberechnungen erfolgen im ersten Schritt auf Matrizen deren Dimensionen ein vielfaches der Zahl vier sind (also z.B. 4x4, 8x8, 16x16) und es werden nur die ganzzahlige Multiplikationen ohne Vorzeichen durchgeführt. Da es viele Ansätze gibt Matrizen zu multiplizieren, wurde eine Einschränkung getroffen. Die Multiplikation erfolgt mit dem Outer-Produkt Ansatz, da dieser Ansatz auch in dem maßgebenden Dokument zu dieser Arbeit beschrieben ist [8].

1.3 Struktur der Arbeit

Zunächst wird im Kapitel Theoretische Grundlagen die Matrix-Multiplikation beschrieben. Es wird detailliert, dass sich die Matrix-Multiplikation in Teile aufteilen und damit Schrittweise abarbeiten lässt.

Nachdem die Matrix-Multiplikation beschrieben ist, wird ihre Relevanz in den Anwendungsbereichen der Neuronalen Netze und der 3D-Computergrafik dargestellt.

Danach werden die momentan ablaufenden Anstrengungen des RISC-V Gremiums zur Erstellung einer Extension zur Matrix-Multiplikation beschrieben. Dabei gibt es drei Ansätze. Der in dieser Bachelorarbeit untersuchte Ansatz wird zusammen mit den anderen Ansätzen vorgestellt. Dabei wird der Strip-Mining Ansatz erläutert und es wird eine Softwareanwendung besprochen, die diesen Ansatz als Computerprogramm implementiert.

Nachdem der Ansatz beschrieben ist, wird der NEORV32 CPU als das Ziel der Implementierung beschrieben. Das Design wird herausgearbeitet und es wird diskutiert welche Änderungen an welchen Stellen vorgenommen werden um den CPU um Matrix-Befehle zu erweitern.

Danach wird der VHDL Quellcode des NEORV32 CPU erweitert und auf einen FPGA programmiert. Auf dem FPGA wird eine Software-Anwendung ausgeführt, die die Matrix-Instruktionen verwendet um eine Matrix Multiplikation durchzuführen.

Schließlich wird das Ergebnis bewertet.

2 Stand der Technik

Ein Komitee hat im Jahre 2015 mit der ersten Version der RISC-V Vector Extension begonnen. Im Jahre 2021 wurde das Release 1.0 ratifiziert und ist jetzt in einen festgehaltenem, unveränderlichen Zustand (Frozen) versetzt worden.

RISC-V wurde durch die bereits ratifizierte RISC-V Vector Extension (RVV) um Register für Vektoren und explizite Anweisungen zur Manipulation dieser Vektoren erweitert.

In der RISC-V Vector Extension Spezifikation [1] wird das Konzept des Strip-Mining beschrieben und hervorgehoben. Die Software lädt den Cache mit einem Teil des Vektors voll und beginnt den Cache dann abzarbeiten, in dem der Teil des Vektors dann wiederum teilweise in den CPU geladen, verarbeitet und danach wieder aus dem CPU zurück in den Speicher übertragen wird. Das Strip-Mining wiederholt diese Abläufe, bis die Aufgabe erledigt ist.

Diese zwei Abläufe (Cache-Frame laden und Register laden) werden dann solange wiederholt, bis der gesamte Vektor abgearbeitet ist. Ein Hinweis auf dieses Strip-Mining Vorgehen findet sich in der RISC-V Vector Extension Spezifikation. Dort gibt es das Beispiel in Kapitel "6.4. Example of stripmining and changes to SEWßum Strip-Mining.

Die Alternative zum Hardware unterstützen Strip-Mining wäre es, jedes einzelne Vektor-Element einzeln in ein Register des CPU zu laden und dann mit einem Element des zweiten Vektors zu verrechnen. Dabei wird für jedes Element ein Laden und Speicher aus dem Speicher notwendig und man bezahlt mit einem hohen Verlust an Geschwindigkeit für die Einfache Implementierung.

Der Vorteil der RISC-V Vector Extension Register ist es, dass diese Register mehrere Elemente parallel auf einen CPU-Takt hin zusammenrechnen können. Der CPU verfährt dann in einem echten SIMD-Betriebsmodus (Single Instruction, Multiple Data) und spart eine vielfaches an Taktzyklen.

Es sind auch komplett neuartige Designs denkbar. Wenn der Cache-Speicher über mehrere Kanäle an den CPU angebunden ist, könnte der CPU die Vektor-Register auch parallel über alle Kanäle befüllen, mehrer Register parallel verrechnen und die Ergebnisse parallel wieder in den Cache zurückliefern.

Diese Prinzip des Strip-Mining wird auch in anderen mathematischen Bibliotheken verwendet. Eine bekannte Bibliothek ist blis [13].

blis verwendet den Begriff kernel oder microkernel. Ein kernel ist Code, der auf einem Cache-Frame ausgeführt wird, als den Code darstellt, den der CPU

sehr schnell ausführen kann. Optimierte Anweisung für verschiedene CPU Architekturen werden hauptsächlich in dem Code verwendet, der dem kernel zuzurechnen ist um Zeit zu sparen.

blis dokumentiert die Matrix-Multiplikation unter dem Stichwort GEMM. GEMM steht für General Matrix Multiply. Der gemm kernel wird für die GEMM Algorithmen verwendet.

Sollte blis für RISC-V portiert werden, ist es sinnvoll, dass sich RISC-V an das Prinzip des strip-mining und der kernel innerhalb eines Cache-Frames hält und das Strip-Mining ermöglicht. Dies wurde für die RISC-V Vector Extension bereits getan.

Für die RISC-V Matrix-Berechnung gibt es zum jetzigen Zeitpunkt keine ratifizierte Extension. Innerhalb der RISC-V Arbeitsgruppen werden unterschiedliche Ansätze diskutiert. Die Ansätze werden im Kapitel zu den theoretischen Grundlagen beschrieben.

Interessanterweise gibt es eine chinesische Firma namens Stream Computing die sich zur Mission gesetzt hat, high-end RISC-V chips zum Einsatz in Rechenzentren zu entwickeln. Bereits im Jahre 2022 konnte Stream Computing ein eigenes Arbeitsergebnis vorweisen. Dieses Arbeitsergebnis wurde von Stream Computing selbst RISC-V Matrix ISA Specification in der Version v0.1 genannt. Die Version v0.5 wurde im Oktober 2024 erstellt. Es handelt sich aber trotz des Namens nicht um die offizielle Version, die das RISC-V Gremium veröffentlichen wird! Stream Computing hat ebenfalls die notwendigen Anpassungen zum LLVM compiler framework beigetragen! Wie tragbar die Ergebnisse sind lässt sich schwer einschätzen ohne eine eingehende Evaluierung des Quellcodes durchgeführt zu haben.

Es ist jedoch interessant zu sehen, welche Innovationskraft im chinesischen Markt liegt. Während sich das RISC-V Gremium wieder auf eine mehrjährige Reise begibt um eine Matrix-Extension zu erzeugen, wie es bereits bei der Vektor-Extension erfolgt ist, hat eine chinesische Firma vermeintlich bereits ein Design und eine Toolchain zu diesem Design erstellt.

Innerhalb des NEORV32 CPU Core Umfeldes gibt es eine Arbeit von Qizal Arsalan [2] bei der der NEORV32 CPU um Matrix Berechnungen erweitert wird. Diese Bachelorarbeit unterscheidet sich allerdings hinsichtlich der Herangehensweise stark von Qizal Arsalans Arbeit. Der NEORV32 CPU besitzt im Design vorgesehene Erweiterungspunkte namens CFS und CFU. Das CFS (Custom Functions Subsystem) erlaubt es Hardware-Beschleuniger in den NEORV32 CPU einzubringen, die über Speicheradressen (Memory Mapping) aus einer Softwareanwendung mit Daten versorgt werden können. CFU (Custom Function Unit) erlaubt es einzelne Instruktionen zu ergänzen um den CPU um kleinschrittige Operationen zu erweitern.

Qizal Arsalan untersucht den Ansatz der Matrix-Multiplikation mit CFS und CFU. In dieser Bachelorarbeit werden Matrix-Instruktionen in den NEORV32 Chip eingebracht, die nicht über das CFS oder CFU Feature funktionieren sondern über die Execution-Engine, also der State-Machine, des NEORV32 Chips. Der Grund ist, dass die Speicheradressabhängigkeit von CFS und

CFU nicht konform mit einer Matrix-Spezifikation des RISC-V Gremiums sein können und nicht sein werden. Bei den Systemen CFS und CFU handelt es sich um Systeme, die nicht Teil der RISC-V Spezifikation sind. Diese Bachelorarbeit versucht die Ergebnisse einer Matrix-Extension im RISC-V Umfeld zu antizipieren und kann daher CFS und CFU nicht verwenden.

3 Theoretische Grundlagen

Dieses Kapitel beschreibt Vektoren und Matrizen aus der Mathematik und deren Berechnung durch ein Computersystem.

Bei der Berechnung ist auf die Geschwindigkeit zu achten. Ausgehend von dem naiven Berechnungsverfahren werden die Schwächen dieses Verfahrens diskutiert und beschrieben, welche Antworten das RISC-V Ökosystem auf diese Probleme hat.

Die ersten Hardwarebeschleunigungen sind mit der RISC-V Vector Extension für Vektoren eingeführt worden. Damit kann ein RISC-V Vektor SIMD unterstützen. Aufbauend auf der Vector-Extension wird diskutiert, welche Ansätze für die Berechnung von Matrizen verwendet werden können.

Der in dieser Arbeit gewählte Ansatz wird beschrieben.

3.1 Vektoren und Matrizen

Ein Vektor ist eine eindimensionale Gruppierung von skalaren Werten. Es handelt sich dabei um eine Fortführung von Skalaren zu neuen Objekten der Mathematik, wie sie auch z.B. bei komplexen Zahlen statt findet. Auf Vektoren sind mathematische Operationen wie Addition definiert, die wiederum Vektoren erzeugen. Es sind auch Operationen wie das Skalarprodukt oder der Betrag definiert die aus Vektoren wieder Skalare erzeugen.

Eine Matrix wird in [12] als rechteckiges Zahlenschemata mit m Zeilen und n Spalten beschrieben.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{d1} & a_{d2} & a_{d3} & \dots & a_{dn} \end{bmatrix}$$

Die Multiplikation zwischen zwei Matrizen A und B ist nicht kommutativ und nur definiert, wenn die inneren Dimensionen der Matrizen übereinstimmen, wenn also die Spaltenzahl der ersten Matrix mit der Zeilenzahl der zweiten Matrix übereinstimmt.

Zur Berechnung der Matrix-Multiplikation

$$C = A * B \quad (3.1)$$

muss zur Berechnung eine Zelle der Matrix C ein Skalarprodukt aus einer Zeile von A und einer Spalte von B berechnet werden. Dabei wird zunächst aus der Spalte von B durch Transponieren eine Zeile gemacht und dann der Zeilenvektor aus A mit dem Zeilenvektor aus B durch das Skalarprodukt in einen Skalar umgerechnet. Dieser Skalar wird dann in die Matrix C eingesetzt.

Ein anschaulichere Beschreibung bietet das Falksche Schema [5].

3.2 Matrixmultiplikation mit SISD

Laut <https://de.wikipedia.org/wiki/Matrizenmultiplikation> verwendet eine naive Implementierung in Software drei For-Schleifen.

```

1 function matmult(A,B,l,m,n)
2
3 // Ergebnismatrix C mit Nullen initialisieren
4 C = zeroes(l,n)
5
6 for i = 1 to l // Schleife ueber die Zeilen von C
7   for k = 1 to n // Schleife ueber die Spalten von C
8     for j = 1 to m // Schleife ueber die Spalten von A /
                          Zeilen von B
9       C(i,k) = C(i,k) + A(i,j) * B(j,k)
10    end
11  end
12 end
13
14 return C

```

Der Nachteil an diesem naiven Verfahren ist nicht sichtbar, wenn man sich den Pseudo-Code oder eine Implementierung in einer realen, höheren Programmiersprache ansieht. Diese naive Implementierung beschränkt sich auf SISD (Single Instruction Single Data) Anweisungen. Wenn der Compiler das Optimierungspotential nicht erkennt, generiert er SISD Anweisungen, die jede Multiplikation und Addition einzeln ausführen.

In beiden Architekturen Van-Neumann und Harvard gilt, dass eine Berechnung nicht im Speicher sondern nur innerhalb der CPU ausgeführt wird. Da die Daten im Speicher stehen muss also eine Übertragung zwischen Speicher und CPU erfolgen. Man nennt dies die Load-Store-Architektur.

Der Nachteil der naiven Implementierung liegt also in der Tatsache begründet, dass Werte nicht in den Registern des CPU verbleiben können, da ein CPU nur wenige Register besitzt und diese Register wiederverwendet werden müssen. Um die Register wiederzuverwenden muss der enthaltene Wert zurück in den Speicher geschrieben werden! Dies ist in der höheren Programmiersprache nicht dargestellt und wird erst sichtbar, wenn man sich den Assembler-Quellcode ansieht, den der Compiler erstellt.

3.3 Matrixmultiplikation mit Strip-Mining und SIMD

Ein Ansatz diesen Verlust, der durch Load-Store entsteht zu minimieren, ist das Strip-Mining in Kombination mit der Verwendung von SIMD. SIMD (Single Instruction Multiple Data) erlaubt es mehr als ein Datum gleichzeitig durch eine einzige Instruktion verarbeiten zu können. Die Addition zweier Vektoren geschieht elementweise und jedes Elementpaar kann unabhängig von allen anderen Paaren in jeglicher Reihenfolge addiert werden. Damit eignet sich die Vektor-Addition für SIMD Anweisungen, wenn der CPU die passende Hardware bereit stellt.

Des weiteren erlaubt das Strip-Mining auch Datenobjekte zu verarbeiten, die so groß sind, dass sie nicht als Ganzes in die Register der CPU passen.

Beim Strip-Mining wird von einer Ressourcen-Hierarchie ausgegangen. Die Ressourcen sind dabei die Register und der Speicher einer CPU, wobei Register letztendlich auch nur sehr kleine Speicherzellen sind. Der Grundgedanke ist, dass je näher Speicher am CPU liegt, das dessen Preis steigt und daher kleiner in der verfügbaren Größe ausfallen muss, aber gleichzeitig auch um einiges schneller als weiter entfernt liegender Speicher ist. Die Hierarchie besteht aus den Register, gefolgt von einem oder mehreren Cache-Leveln und schließlich den RAM Bausteinen.

Die Vektor- oder Matrix-Daten kommen aus dem Speicher und müssen dem CPU in Form seiner Register zugeführt werden. Die Addition zweier Vektoren Beispielsweise kann nur durchgeführt werden, wenn Vektordaten in den Registern der CPU stehen. Die Kunst besteht also darin, die Daten in effizienter Form an den CPU heranzutragen. Beispielsweise ist es ineffizient ständig im Speicher hin- und her zu springen. Ein lineares Laden ist effizienter um die Lokalität der Daten innerhalb des Cache auszunutzen.

Eine Cache-Hierarchy legt nahe, Vektoren blockweise zu laden, so das ein Block jeweils genau in den verfügbaren Cache passt. Einmal geladen, dient der Cache dem CPU als schnellen Speicher und ein Durchgriff auf den langsameren RAM-Speicher erfolgt nur, wenn die Daten im Cache verarbeitet worden sind.

Die Register innerhalb der CPU sind von begrenzter Größe. Eine sinnvolle Annahme ist etwa 1024 bit Registerbreite. Damit passen dort dann 32 Elemente eines Vektors hinein, wenn jedes Element eine 32-Bit Zahl darstellt.

Wenn nun sehr große Vektoren addiert werden sollen, dann kommt Strip-Mining zum Einsatz. Mit groß ist hier gemeint, dass die Vektoren weder im Ganzen in die CPU-Register passen noch passen die Vektoren im Ganzen in einen Cache-Frame. Die Software verwendet eine Instruktion um zu ermitteln, wie viel Platz innerhalb der CPU internen Registers verfügbar ist. Der CPU Antwortet mit der Registerbreite.

Für die Berechnung von Vektoren wurde die RISC-V Vector Extension (RVV) [1] erstellt und ratifiziert. Die Vektor Extension ist für diese Arbeit relevant,

obwohl sie zwar Vektoren verarbeitet aber damit die Grundlage für die Verarbeitung von Matrizen legt.

3.4 Die RISC-V Vector Extension

Ein RISC-V CPU besitzt die sogenannte X-Register-File. Es handelt sich damit um alle Register, die ein RISC-V CPU standardmäßig, ohne Extensions hat. Dies könnten beispielsweise die Register x0 bis x31 sein. Die Länge der Register in bits wird durch die Konstante XLEN festgelegt.

Die RVV erweitert einen RISC-V CPU um zusätzliche Vektor Register v0 bis v31. Die Länge der Vektor Register wird durch die Konstante VLEN (in Anlehnung an die Konstante XLEN) vorgegeben. VLEN wird bei der Synthese des Designs auf einen Wert festgelegt und ist nicht zur Laufzeit änderbar. VLEN stellt somit eine Eigenschaft des CPU dar. Die RVV Spezifikation [1] macht die Vorgabe:

“The number of bits in a single vector register, $VLEN \geq ELEN$, which must be a power of 2, and must be no greater than 2^{16} ”.

Das besondere an den Vector Registern ist, dass sie konfiguriert werden können. Es lässt sich festlegen, welcher Datentyp in den Registern erwartet wird, also 8-bit Bytes, 16-bit Halbwörter oder 32- oder 64-bit Wörter. Zusätzlich lassen sich die Vektor-Register Gruppieren, also aneinanderhängen um noch größere Register zu bilden.

Der Vorteil ist, dass man beispielsweise 8-bit bytes als Datentyp festlegen kann und dann vier Vektor-Register gruppieren kann. Unter der Annahme das ein RISC-V Chip mit einer VLEN von 2^{16} synthetisiert worden ist, ergibt sich damit Speicher für beispielsweise 262144 bytes. Werden zwei Vektoren dieser Länge in den CPU geladen, kann eine Vektor-Addition all dieser Elemente mit nur einer einzigen Instruktion erfolgen! Dies ergibt einen Speedup um den Faktor 262144 gegenüber einem naiven Load-Store Ansatz, bei dem jedes 8-Bit Elementpaar einzeln geladen und verrechnet wird.

Die Verwendung der Vector-Extension erfolgt durch ein Hardware-Software-Co-Design. Damit ist gemeint, dass eine Anwendung mit der Hardware zusammenarbeiten muss um von der Hardware-Beschleunigung zu profitieren. Es ist weder möglich durch Software oder Hardware alleine eine Beschleunigung zu erzielen. Moderne optimierende Compiler wie der GCC für RISC-V oder die LLVM Infrastruktur wurden um das Wissen um die RISC-V Vektor-Extension erweitert und können den Einsatz automatisch erkennen und das gewünschte Hardware-Software Co-Design automatisch generieren!

Dies lässt sich testen, indem man den naiven Matrix Multiplikations Algorithmus in godbolt eingibt und die neuste Version des GCC auswählt. Im generierten Assembler-Code findet man die RVV-Instruktionen wieder!

Das Hardware-Software Co-Design sieht das iterative Vorgehen Strip-Mining vor, bei dem die Software die Hardware in jeder Iteration aufs neue konfiguriert und dann Daten zur parallelen Verarbeitung durchführt.

Die Schritte sind:

1. Die erste bzw. nächste Iteration beginnt.
2. Die Applikation entscheidet, ob es effizienter ist die Vektor Operation mit oder ohne Vektor-Erweiterung auszuführen. Wenn die Vektorlänge beispielsweise nur aus zwei Elementen besteht ist es schneller diese beiden Element einfach so zu verrechnen.
3. Wenn sich die Anwendung dazu entscheidet die Vektor-Extension zu verwenden, dann beginnt die Strip-Mining Schleife. Wenn es zu wenige Elemente gibt, dann springt die Anwendung zu einem Label, dass die Verarbeitung terminiert und berechnet den Rest der Element dort manuell ohne Hardwarebeschleunigungen.
4. Die Anwendung bestimmt den Anteil der Vektor-Elemente, die noch verarbeitet werden müssen. Dieser Wert wird Application Vector Length (AVL) genannt. Beispielsweise entspricht die AVL in der ersten Iteration der gesamten Länge des Vektors. Mit jedem Schleifendurchlauf werden Teile des Vektors verarbeitet und die AVL nimmt immer mehr ab bis schließlich keine Elemente zur Verarbeitung mehr übrig sind und das Verfahren terminiert.
5. Neben der AVL muss die Anwendung auch in jeder Iteration den Datentyp bzw. die Größe der einzelnen Vektorelemente konfigurieren. Die Größe wird (= Selected Element Width, SEW) genannt. Es wird auch bestimmt, wie die Vektorelement in den Registern angeordnet werden und wie die Vektor-Register gruppiert werden. Diese Konfiguration wird Group Modifier oder LMUL genannt. Um die Konfiguration vorzunehmen ruft die Anwendung die Instruktion `set(i)vli` auf.
6. Die Vector Engine verwendet die konfigurierten Werte AVL, SEW und LMUL und berechnet die Batch Größe. Ein Batch ist die Menge an Vektor-Elementen die in einer einzigen SIMD-Operation verrechnet werden. Hier kommt der Hardware-Teil des hardware-software co-designs zum tragen, denn der RISC-V CPU und die enthaltene Vector-Engine müssen entscheiden, wie viele Elemente jetzt tatsächlich parallel verarbeitet werden können. Dabei berücksichtigt die Vector-Engine wieviele Rechenknoten sie zur Verfügung hat um parallel zu rechnen. Die ermittelte Anzahl an Elementen wird vector length (VL) genannt und von der Hardware an die Anwendung zurückgegeben. Dazu gibt die Instruktion `set(i)vli` diesen Wert in ein Rückgaberegister aus, aus dem sich die Anwendung den Wert holen kann.
7. Die Anwendung kennt jetzt die VL, die für diese Iteration gilt. Damit kennt die Software die Batch-Size. Die Software wird nun die Vector-Load Instruktionen (`vle8`, `vle16`, ...) verwenden um einen Anteil des Vektors oder der Vektoren, der genau der VL entspricht in die Vektor-

Register zu laden. Die Anwendung wird auch einen Zeiger nach vorne verschieben, um sich zu merken von welcher Stelle in der nächsten Iteration geladen werden muss.

8. Die Vector-Engine verarbeitet die Vector-Load Anweisungen und schreibt die geladenen Daten in die Vektor-Register.
9. Die Anwendung führt nun eine Instruktion aus um die Vektoren zu verrechnen. Es können beispielsweise eine elementweise Addition (vadd), Subtraktion (vsub), Multiplikation oder sonstige Operationen ausgeführt werden.
10. Die Vektor-Engine führt nun die Instruktion auf allen Elementen parallel aus. Es handelt sich also um eine SIMD-Operation, bei der mehrere Daten aufgrund einer einzigen Anweisung verarbeitet werden. Damit ist der aktuelle Batch der Iteration abgearbeitet.
11. Die Anwendung führt eine Store-Anweisung aus um die Ergebnisdaten aus dem Ergebnisregister in den Speicher zurückzuschreiben und Platz für neue Ergebnisdaten zu schaffen.
12. Die Vektor-Engine verarbeitet die Store-Anweisung und die Daten werden in den Cache bzw. den RAM übertragen.
13. Die nächste Iteration beginnt. Im Grunde springt die Anwendung von hier zurück zum ersten Punkt dieser Auflistung.

Hier alles in Form von Quellcode:

Now that the basic idea and principle has been described in natural language, let's look at code that implements the strip-mining hardware-software co-design.

```
# vector-vector add routine of 32-bit integers      1
#                                                  2
# void vvaddint32(size_t n,                      3
#     const int*x, const int*y, int*z)           4
# {                                              5
#     for (size_t i=0; i<n; i++)                 6
#     {                                          7
#         z[i] = x[i] + y[i];                   8
#     }                                         9
# }                                           10
#                                              11
# Mapping parameters to argument register (a0 .. a7) 12
# a0 = n, a1 = x, a2 = y, a3 = z               13
#                                              14
# Non-vector instructions are indented           15
vvaddint32:                                     16
                                              17
# Set vector length based on 32-bit vectors.      18
# t0 contains the amount of elements that will be 19
# processed                                       20
vsetvli t0, a0, e32, ta, ma                     21
                                              22
# Load for first vector                         23
```

```

vle32.v v0, (a1)
24
25
    # Decrement number of elements that
    # still need processing.
26
27
    # t0 is the result of vsetvli. It
    # is the amount of elements processed
    # this iteration
28
29
    sub a0, a0, t0
30
31
32
    # Multiply number done by 4 bytes (= 32 bit
    # integer) because of SEW=e32
33
34
    slli t0, t0, 2
35
36
    # Increment/Advance pointer for first vector
    add a1, a1, t0
37
38
39
    # Load for second vector
    vle32.v v1, (a2)
40
41
    # Increment/Advance pointer for second vector
    add a2, a2, t0
42
43
44
    # Sum vectors
    vadd.vv v2, v0, v1
45
46
47
    # Store result back to memory
    vse32.v v2, (a3)
48
49
50
    # Increment/Advance pointer for destination
    add a3, a3, t0
51
52
53
    # Loop back to the beginning of the for loop
    # if the element count is not zero. Otherwise
    # continue with the ret instruction
54
55
56
    bnez a0, vvaddint32
57
58
    # Finished with the function
    ret
59
60

```

Jetzt, da die Wirkungsweise der RVV Extension beschrieben worden ist, kann nachvollzogen werden, in welchem Spannungsfeld die RISC-V Gremien Entscheidungen treffen müssen, wenn es um den nächsten Schritt hin zu einer Matrix-Extension geht.

Da die RVV Extension ratifiziert worden ist, gibt es bereits Chips die auf dem Markt verfügbar sind, und die die RVV implementieren. Ein Beispiel ist der OrangePi RV2 Single Board Computer, der einen Spacemit-K1 RISC-V Chip verwendet.

Das bedeutet, Firmen haben bereits Designs erstellt und Investitionen in die RISC-V RVV Strategie investiert. Wenn eine Matrix-Extension Register wiederverwenden würden, hätten solche Firmen einen Vorteil. Auf der anderen Seite möchte man das bestmögliche Verfahren ermöglichen um effizient mit Matrizen rechnen zu können und sich damit von der Konkurrenz abzuheben. Eventuel ist der Ansatz, der für Vektoren gewählt wurde, für Matrizen nicht optimal.

Es gibt daher laut Krste Asanovic [3] bei der Arbeit an einer Extension für Matrizen drei große Strömungen.

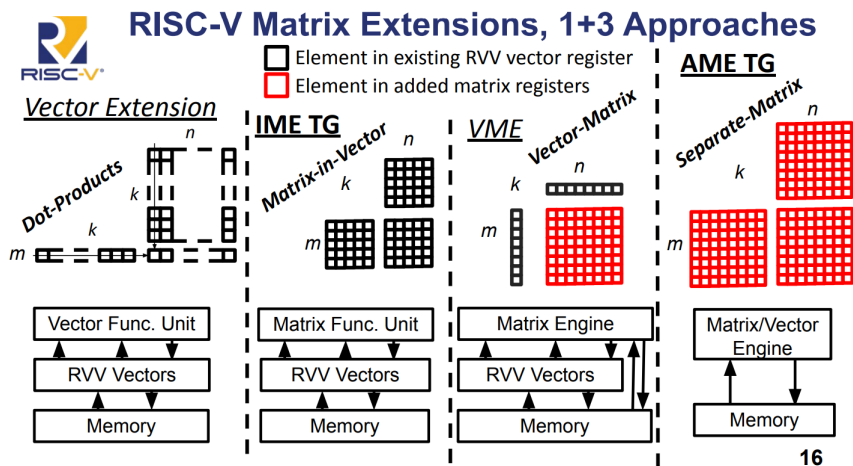


Abbildung 3.1: Ansätze zur Matrix-Extension

Zunächst wird in der linken Spalte der Abbildung 3.4 gezeigt, dass sich eine Matrix theoretisch elementweise berechnen lässt, wenn für jedes Element der Matrix individuell eine Dot-Produkt mit Hilfe der RVV-Extension berechnet wird. Dies ist nur möglich, wenn der CPU die RVV besitzt und gleichzeitig, wenn die Matrix spalten- oder zeilenweise in ein Vektor-Register passt.

Die Integrated Matrix Extension IME [7] verwendet die RVV-Register wieder um darin Matrix-Berechnungen auszuführen. Es wird keine spezielle Hardware für die Matrix-Berechnung hinzugefügt. Die erklärten Ziele des IME-Charter sind es, einen bedeutenden Geschwindigkeitsvorteil zu erzielen, bei vergleichbaren Kosten, wie sie bereits für die RVV anfallen.

Die Vector-Matrix Extension VME baut auf die RVV-Extension auf und führt zusätzlich einen sogenannten Matrix-State bzw. eine Matrix-Engine hinzu. Mit dem Matrix-State ist gemeint, dass der CPU um Komponenten erweitert wird, die in der Lage sind zwischenergebnisse bei einer Matrix-Multiplikation zu speichern. Es kommt bei der VME also weitere Hardware für die Matrix-Berechnung hinzu. Diese zusätzliche Hardware ist in der Abbildung in roter Farbe markiert.

Die Attached Matrix Extension AME geht noch einen Schritt weiter und verwendet dabei keine Vektor-Register aus der RVV-Extension mehr sondern fügt Architectured-State sogar für komplette Matrix Register hinzu. Sowohl die Eingabedaten als auch die Zwischen- und Endergebnisse werden in speziellen Matrix-Registern gespeichert. Damit ist die AME als einziger Ansatz unabhängig von der RVV-Extension und könnte auch komplett ohne RVV-Extension angeboten werden.

Für die offizielle Ratifizierung einer RISC-V Matrix-Extension ist zum jetzigen Zeitpunkt nicht klar, welcher Ansatz verwendet wird oder ob eventuel sogar mehr als eine Spezifikation ratifiziert werden wird.

In dieser Bachelor-Arbeit wird ein Ansatz verfolgt, der sich vom Architectured State her am ehesten an der Attached Matrix Extension AME orientiert. Zu-

stätzlich werden Instruktionen vorgesehen, wie sie bei der Vektor-Extension zu sehen sind. Damit ist gemeint, dass die hinzugefügten Instruktionen den Strip-Mining Ansatz ermöglichen. Damit kann eine Matrixmultiplikation auch für sehr große Matrixen, die nicht als Ganzes in die CPU-Register passen, erfolgen.

3.5 Blockweiße Matrix Multiplikation

Im Folgenden wird das blockweiße Vorgehen erläutert, mit dem die Matrix-Multiplikation ausgeführt werden wird.

Bei der herkömmlichen Matrix-Multiplikation über das Dot-Product von Zeilen und Spalten werden die beiden Matrizen A und B in ihre kompletten Dimension betrachtet, da jeweils komplette Zeilen und Spalten verrechnet werden um ein Matrix-Element zu bestimmen.

Bei sehr großen Matrizen ist dies nicht zielführend, da der CPU die Matrix nur schrittweise erfassen könnte und damit nur sehr ineffizient berechnen kann. Aus diesem Grund bietet sich das Outer Product an.

Die angestrebte Verbesserung besteht darin, eine Matrix blockweise zu berechnen. Damit eröffnet sich, ähnlich wie bei Vektoren, das Potential der parallelen Berechnung durch SIMD Instruktionen. Wie bei Vektoren ist es durch das blockweiße Vorgehen auch für die Berechnung von Matrizen egal in welcher Reihenfolge die einzelnen Ergebnisse berechnet werden. Damit ist auch eine vollkommene Parallelität denkbar.

Als Ausblick würde das für ein RISC-V Design bedeuten, dass ein RISC-V CPU aus mehreren Harts synthetisiert wird, wobei jeder Hart eine Matrix-Engine erhält, also in der Lage ist hardwarebeschleunigt die Matrixmultiplikation zu berechnen. Dann wird eine große Matrix auf alle Cores verteilt um das Ergebnis in einzelnen Blöcken zu berechnen und zurück in den RAM-Speicher zu schreiben.

Zunächst wird anhand eines Beispiels verdeutlicht, was es bedeutet, wenn von blockweißer Berechnung gesprochen wird.

A				B				E				F			
C				D				G				H			
	1	2	3	4				17	18			19	20		
	5	6	7	8				21	22			23	24		
	9	10	11	12				25	26			27	28		
	13	14	15	16				29	30			31	32		

Abbildung 3.2: Matrix Multiplikation blockweise

In Abbildung wird die grüne Matrix mit der blauen Matrix multipliziert. Es werden einzelne Blöcke A bis H innerhalb der beiden Matrizen definiert.

Zur Gegenprüfung ist das Ergebnis in Abbildung ?? dargestellt.

	250	260	270	280
	618	644	670	696
	986	1028	1070	1112
	1354	1412	1470	1528

Abbildung 3.3: Matrix Multiplikation Ergebnis

Dieses Ergebnis wird den Berechnungsvorschriften aus 3.4 zufolge berechnet. Beispielsweise gilt für den grauen Block die Vorschrift $A * E + B * G$

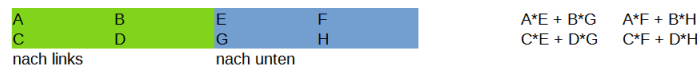


Abbildung 3.4: Matrix Multiplikation Berechnungsvorschrift

Der grau markierte Block ist das Ergebnis der Multiplikation der Blöcke A, E, B und G. Es ist zu prüfen, dass die Multiplikation der Blöcke tatsächlich den grauen Bereich der Ergebnismatrix ergibt.

In Abbildung 3.5 wird der grau hervorgehoben Bereich der Berechnungsvorschrift zufolge berechnet. Zunächst werden die Blöcke A mit E und B mit G zusammen sortiert. Das Ergebnis der beiden Multiplikationen ist in der folgende Zeile zu finden. Das Ergebnis der Addition der Zwischenergebnisse in der untersten Zeile. Die unterste Zeile enthält den grauen Block.

A		E		B		G	
1	2	17	18	3	4	25	26
5	6	21	22	7	8	29	30
A * E				B * G			
59	62			191	198		
211	222			407	422		
A * E + B * G							
250	260						
618	644						

Abbildung 3.5: Matrix Multiplikation Blöcke A, E, B und G

Mit dem Wissen über die blockweise Aufteilung ist es nun möglich per SIMD zu parallelisieren oder Strip-Mining auszuführen, wenn keine parallelen Rechenknoten verfügbar sind. In dieser Arbeit werden Instruktionen analog zur RVV bereitgestellt um das Strip-Mining für Matrizen zu ermöglichen und dabei blockweise vorzugehen.

Es ist noch zu klären, wie die einzelnen Blöcke miteinander multipliziert und addiert werden.

3.6 Outer-Produkt

Das Outer-Produkt wird verwendet um die Matrix-Multiplikation von Blöcken auszuführen. Die Eigenschaft, die das Outer-Produkt nützlich macht ist, dass es die Matrix-Multiplikation auf die Verrechnung von Vektoren reduziert. Damit lässt sich das Verfahren einfach implementieren.

In der Übersicht [8] von Earl A. Killian wird das Outer-Product auf Seite 53 beschrieben.

TODO: Reproduce the definition

Im Grunde kann man an der Formel aus Abbildung 3.4 bereits erkennen, dass es nicht ausreicht Multiplikationen auszuführen, vielmehr müssen Ergebnisse auch aufaddiert werden. Daher ist es notwendig im Design einen State vorzusehen, der Schrittweise durch Addition vervollständigt wird, bis das Endergebnis vorliegt.

Dieser "Matrix-Kern" ist in Abbildung 3.6 aus [8] dargestellt. Dieses Konstrukt wird auch Matrix Accumulator MAC genannt, da er Zwischenergebnisse aufsummiert, also kumuliert.

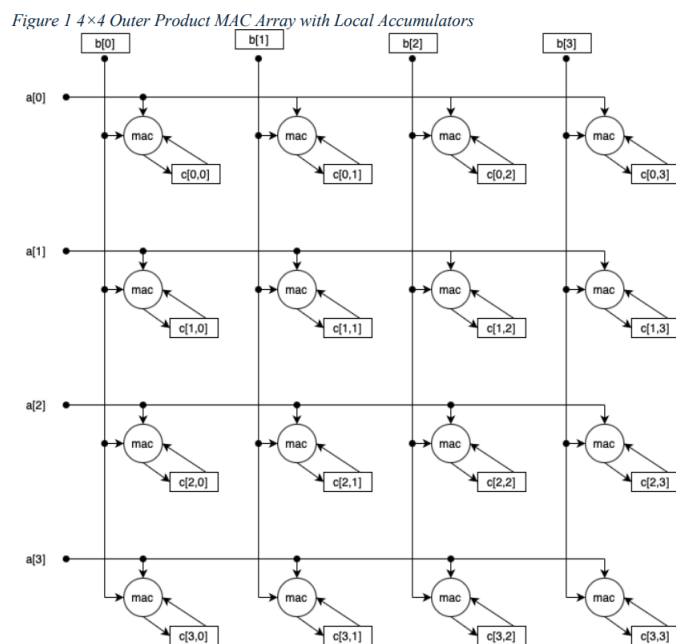


Abbildung 3.6: Matrix Accumulator (MAC)

Wenn zwei Vektoren außen an den MAC angelegt werden, dann berechnet er ein Zwischenergebnis für das Outer Product aus den beiden Vektoren und akkumuliert das Zwischenergebnis zum Rest auf. Wenn alle Vektoren angelegt worden sind, enthält der MAC somit das komplette Outer Product.

Die Kombination aus dem blockweißen Ansatz und dem Outer Product mittels MAC ermöglicht es ein Design zu erstellen, dass dem angestrebten RISC-V AME Ansatz ermöglicht.

4 Durchführung und Methodik

Das Ziel ist beschrieben. Es geht um die Erweiterung eines RISC-V Kerns um eine hardware beschleunigte Matrix-Multiplikation. Dabei soll die Arbeit möglichst nahe an einer der möglichen Erweiterungen liegen, die momentan tatsächlich von den RISC-V Gremien vorbereitet werden. Die theoretischen Grundlagen wurden aufgezeigt.

In diesem Kapitel soll es um die praktische Umsetzung der bis hierher erarbeiteten Punkte gehen. Dazu wird der RISC-V Kern analysiert. Wenn die Bestandteile beschrieben sind, dann werden die möglichen Punkte an denen eine Anpassung erfolgen könnte bestimmt und durch Abwägungen bestimmt, wo die Änderungen tatsächlich eingebracht werden.

Es soll auch beschrieben werden, wie Software für das Hardware- Software-Co-Design erstellt wird. Da die Matrix-Multiplikation kein absolut triviales Programm darstellt, ist die Aufgabe der Software-Entwicklung nicht zu vernachlässigt.

4.1 Der NEORV32 Core und dessen Einsatz

Der NEORV32 Chip ist ein VHDL- und auch ein Verilog-Design von Stephan Nolting und anderen Programmierern¹, welches als Open-Source Projekt unter der BSD-3 Lizenz veröffentlicht ist [10]. Es gibt es sehr gute Dokumentation zu dem Projekt in Form von textueller Dokumentation, dem sogenannten Datenblatt [11]. Zudem ist auch der Quellcode selbst sehr gut dokumentiert und sogar einheitlich formatiert und somit gut lesbar.

Zunächst ist zu sagen, dass man den VHDL- oder Verilog-Quellcode auf unterschiedliche Arten nutzen kann. Dabei sind die unterschiedlichen Möglichkeiten interessant für unterschiedliche Phasen eines Projekts.

Zur Durchführung wird der Quellcode zunächst durch einen Simulator evaluiert. Die Evaluierung ermöglicht dann die Simulation des Designs für einen bestimmten Zeitschritt, beispielsweise für eine Millisekunde. Die Simulation wird mit Eingangssignalen "stimuliert". Das Design verarbeitet dann die eingehenden Signale und der Simulator speichert die Signaländerungen in .vcd Dateien. Ein sogenannter Waveform-Viewer kann die Signale dann anzeigen. Durch visuelle Inspektion kann nun geprüft werden, ob das Design so auf die Eingabedaten reagiert, wie vom Designer erwartet. Das NEORV32 Projekt bietet ein vorgefertigtes Skript, welches den Simulator GHDL aus-

¹<https://github.com/stnolting/neorv32/graphs/contributors>

führt. Also solches ist das NEORV32 Projekt für Anfänger sehr gut geeignet, da es diese Funktion bereits automatisch bereitstellt.

Wenn die Simulation die erwarteten Ergebnisse zeigt, dann kann ein Schritt weiter gegangen werden. Das Design wird nun Synthetisiert statt Evaluert. Die Synthese-Werkzeuge führen das sogenannte "Lowering" durch. "Lowering" im Sinne des Wortes soll andeuten, dass die Synthese-Werkzeuge eine Transformation von den abstrakten, also hohen Beschreibungen im Quellcode in Designs aus physischen Logikzellen durchführen. Von Abstrakt zu physisch findet also ein "Lowering" statt. Sind die Logikzellen bestimmt, lokalisiert und miteinander verbunden, kann diese Aufteilung durch eine Bitstream-Datei auf einen FPGA übertragen werden.

Um die Synthese-Werkzeuge einzusetzen gibt es ein Parallelprojekt zum NEORV32 namens, neorv32-setups. Innerhalb dieses Projekts, befindet sich ein TCL-Script. Dieses TCL-Script kann in der Entwicklungsumgebung Vivado ausgeführt werden und erzeugt dann ein Vivado-Projekt. Wenn dieses Vivado-Projekt dann schließlich in Vivado geöffnet wird, dann kann das Projekt übersetzt werden. Dabei führt Vivado seine gesamte Synthese Toolchain durch und führt damit das Lowering konkret aus. Das Ergebnis ist die bitstream-Datei. Vivado kann mit dem FPGA Board ARTY A7 100T sprechen und die bistream Datei über eine USB Verbindung auf den FPGA übertragen. Der FPGA wird sich dem bistream zufolge konfigurieren und ab dann läuft der NEORV32 CPU auf dem FPGA, also ob er in Silikon gegossen worden wäre.

Das Vorgehen in dieser Arbeit ist es zunächst die Änderungen in das NEORV32-Design einzubringen und zu simulieren, bis die Simulation erfolgreich ist. Danach erfolgt die Synthese mit Vivado. Erfahrungsgemäß verhält sich das synthetisierte Design anders als das simulierte. Die Arbeit ist also nach der Simulation nicht getan.

4.2 Architektur des NEORV32 Core

Der NEORV32 besitzt folgenden, in Abbildung 4.1 dargestellten Aufbau.

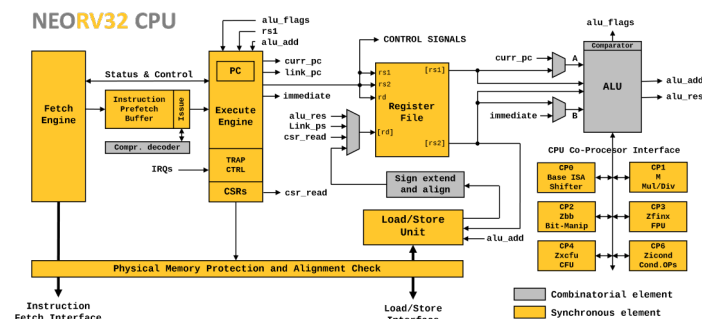


Abbildung 4.1: NEORV32 Architektur

Er ist im groben aus einem Frontend und einem Backend aufgebaut. Das

Frontend ermittelt die nächste Anweisung und das Backend führt diese Anweisung dann aus. Beide Teile arbeiten in gewisser Weise unabhängig voneinander. Damit kann das Frontend bereits Instruktionen puffern während das Backend Abarbeitungen macht. Im Falle von Sprüngen kann sich der Puffer dann mit den neuen Anweisungen am Sprungziel füllen, während das Backend parallel arbeitet.

4.3 Die Fetch-Engine

Die Fetch-Engine steht am Anfang der Abarbeitung einer Anweisung. Sie ist für das Laden der Anweisung aus dem Speicher zuständig und lädt von der Adresse, die momentan in Program Counter-Register steht. Die Fetch Engine arbeitet unabhängig vom Rest des CPU. Sie implementiert intern eine State-Machine, die Anweisungen puffern kann und sie kann intern Anweisungen laden ohne, dass die Anweisungen akut vom Rest des Systems gebraucht wird.

Aus diesem Grund gibt es ein weiteres Signal, welches in Abbildung 4.2 als "debug_valid_i" sichtbar gemacht wurde und angibt, ob die geladene Instruktion gültig ist und verwendet werden sollte oder nicht.

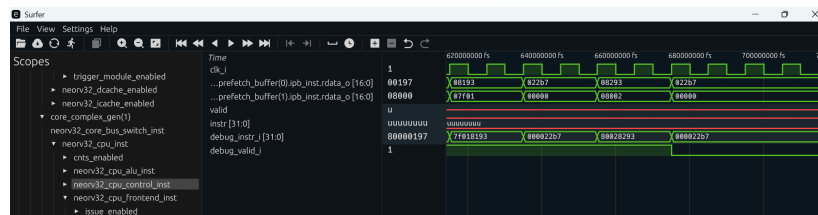


Abbildung 4.2: NEORV32 Gültige Instruktion

Dieses Wissen ist wichtig, wenn man das Verhalten des CPU anhand einer WaveForm-Ansicht diagnostizieren möchte. Dabei geht man oft von einem Assembler-Programm aus, das vom CPU ausgeführt wird. Wenn die Fetch-Engine kreuz und quer Instruktionen lädt, muss man wissen, ob die momentan anliegende Instruktion gültig ist oder nicht.

Es ist des Weiteren zu bemerken, dass die Fetch Engine nur über einen Cache verfügt wenn dies konfiguriert wird. In der Standardkonfiguration ist kein Cache für Instruktionen konfiguriert, wie in der Abbildung 4.1 dargestellt. Die Fetch Engine greift direkt auf den Speicher zu.

4.4 CPU-Control und Execution-Engine

Die Entität "CPU-Control" enthält die Kontrolllogik des CPU. Die CPU-Control enthält den Program-Counter der auf die nächste Anweisung zeigt,

die "Configuration and Status Register" (CSR) und Funktionen für Interrupts und Exceptions.

Bei einigen CPU-Designs für FPGAs findet man eine Kontrollogik vor. Die Kontrollogik besteht oft aus kombinatorischer Logik und gibt damit einfach direkt Signale aus, die ohne Zustand also ohne Speicherelemente direkt aus den Eingabesignalen berechnet werden. Ein Beispiel ist das Design aus [6]. Allgemein wird die Kontrollogik Teile des CPU an- und abschalten. Damit konfiguriert sich ein CPU für jede Anwendung passend neu um diese Anweisung ausführen zu können.

Der NEORV32 CPU ist an dieser Stelle eine Ausnahme. Er besitzt anstatt kombinatorischer Logik eine State-Machine, die Execution-Engine genannt wird, um den CPU zu kontrollieren. Die Execution-Engine ist im inneren eines VHDL Prozesse definiert. Die allermeisten Prozesse enthalten das Reset-Signal und das Clock-Signal in ihrer Sensitivitätsliste. Der Prozess für die Execution-Engine ist anders. Hier befindet sich kein Clock-Signal in der Sensitivitätsliste sondern alle Signale, die die Execution-Engine beeinflussen müssen explizit aufgelistet werden! Dies wird wichtig, wenn die Execution-Engine später erweitert wird. Beispielweise muss die Execution-Engine nachdem sie einen Request nach außen gibt, auch wieder auf diesen Response warten. Das Response-Signal muss in die Sensitivitätsliste aufgenommen werden.

Der Start-Zustand ist EX_RESTART state. Aus dem Reset geht die State-Machine in EX_BRANCHED und erst dann in den Leerlaufzustand EX_DISPATCH über. Der Umweg über EX_BRANCHED erlaubt es auf die ein Instruction Fetch zu warten.

Im Leerlaufzustand EX_DISPATCH, wartet die Execution Engine bis ein Trap oder eine Exception eintritt oder bis das Frontend eine gültige Instruktion meldet, die dann abgearbeitet werden muss. Für normale Anweisungen wird der Zustand EX_EXECUTE betreten.

In EX_EXECUTE wird der nächste Zustand abhängig von der Art der Anweisung gewählt. Das heißt EX_EXECUTE ist ein Switch-State der in mehrere Äste der State-Machine verzweigen kann. Es gibt Zweige für ALU-Operationen, Sprünge und Speicherzugriffe (Load und Store).

Für länger laufende Befehle muss die Execution-Engine auf den Abschluss der Operation warten. Daher sind lange laufende Operationen jeweils durch ein Paar aus Request und Response Zuständen modelliert. Der Request Zustand wird direkt in Richtung Response-Zustand verlassen. Der Response-Zustand wird erst in Richtung EX_DISPATCH verlassen, wenn das eingehende Response-Signal aus der Sensitivitätsliste aktiviert worden ist. Im Response-Zustand werden die erzeugte Daten betrachtet und Signale zurückgesetzt.

4.5 Die Load-Store-Engine

neorv32_cpu.vhd.

Die State-Machine der Execution-Engine produziert eine Vielzahl an Signalen, die andere Entitäten des VHDL-Designs aktivieren und mit Eingabedaten versorgen. Ein wichtiger Teil des Designs ist die sogenannte Load-Store Unit. Die Load-Store Unit hat zur Aufgabe Bus-Requests zu erstellen und Bus-Responses zu verarbeiten.

Die Bus-Requests werden auf dem Bus innerhalb des NEORV32 CPU ausgeführt. Ein Teilnehmer am Bus ist z.B. der Datenspeicher. Mit Datenspeicher ist der Teil des Speichers gemeint, der die Werte der Variablen speichert, also konkret keinen Quellcode sondern eben Daten enthält. Der Datenspeicher wird über Load-Anweisungen gelesen und produziert damit Daten für die Register oder er wird durch Store-Anweisungen geschrieben. Die Aufgabe der Load-Store-Unit besteht darin die Signale der Execution-Engine in passende Bus-Requests für Load- oder Store Anweisungen zu übersetzen.

Dabei ist folgende Besonderheit zu beachten. Die Bus-Requests, die von der Load-Store-Unit erzeugt werden sind in der Datei neorv32_top.vhd mit dem Data-Cache verbunden. Damit unterstützt der NEORV32 CPU eine Cache-Hierarchie für Daten. Anstatt direkt auf den Speicher zuzugreifen, greift die Load-Store-Unit immer zunächst in den Cache. Wenn ein Cache-Hit stattfindet, ergibt sich ein Geschwindigkeitszuwachs.

TODO: Execution Engine, Erweiterung um neue Pfade für Matrix Engine

TODO: LSU und Cache

TODO: Softwareerstellung (Intrinsics)

5 Erweiterung des NEORV32-CPU

In diesem Kapitel werden die Erweiterungen an den Entitäten des vorgegebenen Designs beschrieben, die gemacht wurden um die Matrix-Multiplikation zu ermöglichen.

5.1 Die Matrix-Register

TODO: erkläre die Erweiterung im die Matrix-Register.

5.2 Die Instruktionen

Um die Matrix-Multiplikation durchzuführen sollen neue Instruktionen hinzugefügt werden, die es noch nicht bereits in anderen Extensions für RISC-V gibt. Die Instruktionen sollen das Strip-Mining ermöglichen und sich damit an der RVV-Extension orientieren.

Bei der Diskussion des Strip-Mining bei der RVV-Extension wurde die Instruktion `set(i)vli` besprochen, mit der die Vektor-Engine um Datentypen und gewünschte Datenlängen konfiguriert wurde. Des weiteren wurden die Instruktionen `vle32.v`, `vadd.vv` und `vse32.v` im Rahmen des Beispiel-Codes verwendet. `vle32.v` lädt Daten aus dem Daten-Speicher in die Vector-Register (Load-Operation), `vadd.vv` ist die SIMD Instruktion, die zwei Vektoren elementweise parallel miteinander addiert hat und `vse32.v` speichert die Daten schließlich in den Daten-Speicher zurück (Store-Operation)

Für die Matrix-Extension werden die Instruktion `mle32`, `mmul32` und `mse32` hinzugefügt. Dabei ist `mle32` die Load-Operation, `mmul32` die SIMD-Instruktion und `mse32` die Store-Operation. Ein Äquivalent zur `set(i)vli` ist nicht vorgesehen, da die Matrix-Engine zum jetzigen Stand keine Konfigurationsmöglichkeiten anbietet.

Der NEORV32 besitzt eine Menge an Beispielen, die sich im Software-Unterordner befinden. Jedes Beispiel kann von Funktionen Gebrauch machen, die über die `neorv32.h` Header-Datei eingebunden werden können. Teil dieser Funktionen sind sogenannte Intrinsics. Ein Intrinsic ist eine Funktion, die in der Programmiersprache C geschrieben ist und die Programmiersprache C damit um Funktionalität erweitert, die thematisch zu einem anderen Themengebiet gehört.

Ein Beispiel dafür sind die Intrinsics the X-Window Systems [4]. Hier wird eine einfache zu verwendende Schnittstelle zur Softwareentwicklung mit dem X-Window System für die Programmiersprache erstellt.

Die Intrinsics des NEORV32 sind eine Erweiterung der Programmiersprache C um RISC-V Instruktionen aus C-Quellcode heraus auszugeben. Ein Beispiel für die Verwendung ist das folgende Test-Programm.

```
#include <neorv32.h> 1
2
uint32_t matrix_a[16] 3
    = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 }; 4
5
uint32_t matrix_b[16] 6
    = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 }; 7
8
int main() { 9
10
    uint32_t *matrix_a_addr = &matrix_a[0]; 11
    uint32_t *matrix_b_addr = &matrix_b[0]; 12
13
    uint32_t matrix_unit_a = 0; 14
    uint32_t matrix_unit_b = 0; 15
16
    matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, 17
        matrix_a_addr, 0b010, 0b1011011);
    matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, 18
        matrix_b_addr, 0b010, 0b1011011);
19
    uint32_t matrix_mul = CUSTOM_INSTR_I_TYPE(0x000, 20
        matrix_a_addr, 0b100, 0b1011011);
21
    matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, 22
        matrix_a_addr, 0b001, 0b1011011);
    matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, 23
        matrix_b_addr, 0b001, 0b1011011);
24
    for (int i = 0; i < 16; i++) { 25
        neorv32_uart0_printf("After Matrix A %d\r\n", 26
            matrix_a_addr[i]);
    } 27
    for (int i = 0; i < 16; i++) { 28
        neorv32_uart0_printf("After Matrix B %d\r\n", 29
            matrix_b_addr[i]);
    } 30
31
    return 0; 32
} 33
```

In der Zeile 17 ff wird als Intrinsic "CUSTOM_INSTR_I_TYPE" für I-Type RISC-V Instruktionen verwendet. Aus jeder Zeile mit einem "CUSTOM_INSTR_I_TYPE" Aufruf wird eine RISC-V Assembler Instruktion generiert.

Die Verwendung ist schwer zu verstehen, daher wird jetzt beschrieben, wie die Parameter zum Intrinsic zu interpretieren sind. Der letzte Parameter ist der Op-Code. Für die Matrix-Extension wurde in dieser Arbeit der Op-Code 0b1011011 gewählt. Innerhalb dieses Opcodes werden dann durch

den vorletzten Parameter die drei Anweisungen mle32, mmul32 und mse32 kodiert.

Es ist noch abschließend zu erklären, weshalb Intrinsics verwendet werden müssen. Der grundsätzliche Ablauf bei der Verwendung der Programmiersprache C ist, dass ein Compiler den Quellcode der höheren Programmiersprache in Eigenregie in Assembler-Code übersetzt. Dabei wählt er die Anweisungen selbst aus. Optimierende Compiler führen diese Aufgabe in vielen Fällen besser aus, als es ein Mensch könnte. Im Fall dieser Arbeit ist dieses Vorgehen nicht möglich. Der C-Compiler kennt die neu definierten Instruktionen mle32, mmul32 und mse32 schlicht nicht und wird sie daher nur ausgeben, wenn man den Compiler explizit dazu anweist. Diese explizite Anweisung passiert durch die Intrinsics.

Intern sind die Intrinsics durch Inline Assembly implementiert. Das bedeutet, dass dem C-Compiler exakt vorgegeben wird, welchen inline assembly Code er auszugeben hat. Im Beispiel-Code werden die Instruktionen mle32, mmul32 und mse32 per Opcode generiert.

Im Anhang dieser Arbeit wird die Erstellung eines Compilers beschrieben, der im Zuge dieser Arbeit erstellt worden ist und auch dazu verwendet werden kann, die neuen Instruktionen auszugeben. Der im Anhang beschriebene Compiler wurde verwendet, bis der Weg über die Intrinsics gefunden wurde. Die Intrinsics erlauben es State-of-the-Art Compiler wie den GCC zu verwenden. Dies ist sehr viel komfortabler als einen eigenen Compiler zu pflegen.

5.3 Erweiterung der Execution-Engine

Bis zu diesem Punkt liegt ein Beispiel-Programm mit den neuen Matrix-Instruktionen vor, welches in den NEORV32 CPU geladen und ausgeführt werden kann.

In diesem Abschnitt wird die Execution-Engine erweitert, so dass sie die neuen Instruktion auch ausführt, wenn sie von dem Frontend präsentiert werden.

Als erstes wird der neue Matrix-Opcode aus der Erkennung illegaler Opcodes ausgeschlossen.

```
-- MATRIX EXTENSION                                1
when opcode_matrix_c =>                               2
    illegal_cmd <= '0';                               3
```

Der ALU wird beigebracht, dass sie den Operand B als immediate verwenden muss, sobald sie den Matrix-Extension Opcode sieht. Der Grund ist, dass die Load- und Store-Matrix-Anweisungen die Register-Id als immediate Wert verwenden um zu ermitteln, welches Matrix-Register Sie lesen und schreiben

sollen. Damit muss die ALU diesen Immediate-Wert weitergeben, sonst ist er verloren und kann nicht mehr verwendet werden.

```

-- ALU operand B: is immediate? --
case opcode is
  -- WBI: added matrix opcode here because the mle32,
    mse32 instructions contain an immediate
  -- that needs to be added to the register encoded in
    the instruction
  when opcode_matrix_c | opcode_alui_c | opcode_lui_c
    ...
    ctrl_nxt.alu_opb_mux <= '1';
  when others =>
    ctrl_nxt.alu_opb_mux <= '0';
end case;

```

Im EX_DISPATCH Zustand wird die Matrix Multiplikation gestoppt.

```

matrix_mul_o <= '0';

```

Der EX_EXECUTE Zustand wird um die Behandlung der Matrix-Instruktionen erweitert.

```

-- Matrix Extension --
when opcode_matrix_c =>

case funct3_v is

  -- load from memory into matrix unit --
  when funct3_mle32_c =>
    -- go to the specific states for the matrix
      extension.
    -- The states are inserted in order to load all
      the elements of a matrix in a loop
    exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

  when funct3_mse32_c =>
    exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

  when funct3_mmul32_c =>
    -- go to the state that process matrix operations
    exe_engine_nxt.state <= EX_MATRIX_OPERATION_REQ;

  when others => -- undefined

end case;

```

Für die drei Instruktionen mle32, mse32 und mmul32 werden neue Zustände in die Execution Engine eingefügt, so dass diese Zustände aus EX_EXECUTE angesprungen werden können.

Für die Matrix Load- und Store-Instruktionen wird der Zustand EX_MATRIX_MEM_REQ hinzugefügt.

```

-- trigger a loop of memory requests for the matrix

```

```

when EX_MATRIX_MEM_REQ =>
    -- tell the matrix unit to come online and process
    the request
    ctrl_nxt.lsu_m_en    <= '1';
    if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
        exc_iaccess_c)) = '0') then
        -- memory request if no instruction exception
        -- tell the load store unit to process a request
        ctrl_nxt.lsu_req    <= '1';
        -- forward the immediate from the instruction
        matrix_immediate_o <= immediate;
        -- rs1 (source register 1) carries the address to
        load from / store to
        matrix_mar_load_store_o <= rf_rs1_i;
        exe_engine_nxt.state <= EX_MATRIX_MEM_RSP;
    else
        -- On Error
        exe_engine_nxt.state <= EX_DISPATCH;
    end if;

```

Im Zustand EX_MATRIX_MEM_RSP wird auf die Rückmeldung gewartet. Wenn die Rückmeldung anhand des Signals "lsu_wait_i" reif ist, dann wird der geladene Wert über das write-back signal (wb) in die Register-File (RF) geschrieben. Dann wird die Matrix-Engine deaktiviert.

```

when EX_MATRIX_MEM_RSP => -- wait for memory response
    if (lsu_wait_i = '0') then
        -- write-back to RF if read operation (won't
        happen in case of exception)
        ctrl_nxt.rf_wb_en    <= (not ctrl.lsu_rw) or ctrl
        .lsu_rvs or ctrl.lsu_rmw;
        -- tell the matrix unit to stay offline
        ctrl_nxt.lsu_m_en    <= '0';
        exe_engine_nxt.state <= EX_DISPATCH;
    end if;

```

Für Matrix Operationen gibt es die beiden neuen States EX_MATRIX_OPERATION_REQ und EX_MATRIX_OPERATION_RSP. Mit Matrix Operationen ist die Matrix-Multiplikation gemeint. Da die Matrix-Multiplikation nicht in einem einzigen Schritt ausgeführt ist, muss die Execution-Engine auf die Matrix-Engine warten. Daher ergibt sich die Notwendigkeit für einen Request- und einen Response-State.

```

when EX_MATRIX_OPERATION_REQ => -- matrix add operation

```

```

if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto 3
    exc_iaccess_c)) = '0') then
    matrix_mul_o <= '0';

    case funct3_v is
        when funct3_mmul32_c =>
            matrix_mul_o <= '1';

        when others => -- undefined

    end case;

    matrix_immediate_o <= immediate;

    -- next state
    exe_engine_nxt.state <= EX_MATRIX_OPERATION_RSP;

else
    exe_engine_nxt.state <= EX_DISPATCH;
end if;

when EX_MATRIX_OPERATION_RSP =>

    -- next state
    if (matrix_operation_wait_i = '0') then

        matrix_mul_o <= '0';

        exe_engine_nxt.state <= EX_DISPATCH;

    end if;

```

Diese beiden Zustände sind sehr simple. Im Request-State wird das Signal `matrix_mul_o` aktiviert und in en Response-State übergegangen. Im Response-State wird auf das Signal `matrix_operation_wait_i` gewartet und dann die Matrix Multiplikation deaktiviert.

5.4 Erweiterung der Load-Store-Unit

Der nächste Schritt ist die Erweiterung der Load-Store-Unit. Die Load-Store-Unit ist innerhalb der CPU-Entity definiert.

Die Aufgabe besteht darin, Daten aus dem Datenspeicher in die neuen Matrix-Register zu laden. Hier wäre es möglich die Matrix-Engine als eigenen Bus-Teilnehmer an den Bus zu binden. Die Matrix-Engine hätte dann die Möglichkeit Daten aus dem Datenspeicher über den Bus zu laden.

Der Nachteil daran ist, dass die Matrix-Engine dann Daten lädt, die die Load-Store Unit dann nicht in ihrem Cache geladen hat. Die Load-Store-Unit ist verantwortlich für den Rest des CPU. Damit hätte der CPU dann insgesamt veraltete Daten. Nach Ausführung einer Matrix- Load-Store-Operation müsste die Load-Store-Unit ihren Cache invalidieren, da dieser veraltete Daten enthalten könnte. Daraufhin muss sich der Cache erst wieder aufwärmen und die Performance leidet folglich.

Aus diesem Grund wurde das Laden der Matrix-Engine in die LSU integriert. Wenn eine Matrix Operation eine Matrix berechnet hat und diese Daten dann wieder in den Datenspeicher übertragen werden, dann durchlaufen die Daten den Cache der LSU und wärmen den Cache damit für den Rest des CPU mit aktuellen Daten. Das bedeutet, dass der Rest der ausgeführten Anwendung dann direkt auf die gecachte Matrix zugreifen kann anstelle die Matrix erst wieder aus dem Datenspeicher laden zu müssen.

Damit werden Daten über die Load-Store-Unit geladen, damit die Daten im Cache landen. Die Load-Store-Unit erzeugt Bus-Anfragen, wenn Sie Daten laden und speichern soll. Da ein Matrix-Register aus 16 Elementen besteht, werden beide Prozess, für Load und für Store erweitert.

Der Load-Prozess wird für Matrix-Load-Anweisungen um eine State-Machine erweitert. Der State ML_IDLE deaktiviert die Load-Operation. Der Zustand ML_INIT setzt den Zähler auf 0 und geht in den ML_PROCESS Status über. Im ML_Process Status erfolgen 16 Load Operationen um das Matrix-Register zu füllen. Alle 16-Operation durchlaufen den Cache was positiv ist. Der ML_INCREMENT wird abwechselnd mit ML_PROCESS ausgeführt und zählt den Zähler nach oben.

Sind die 16 Operationen ausgeführt, geht die State-Machine zurück nach ML_IDLE.

Ein etwa gleichartiges Vorgehen wird auch für die Matrix-Store-Operation gewählt, welche 16 Element zurückschreiben muss.

Eine wirkliche Neuerung ist, dass ein neuer Prozess eingefügt wird. Es handelt sich um den PROC_RAM Prozess. Abhängig von den Zählern, die in den Operationen Load und Store verändert werden, produziert der PROC_RAM Prozess Signale, die die Load-Store-Unit verlassen und im CPU mit dem RAM für die Matrix-Engine verbunden sind. Damit wird der RAM der Matrix-Engine gesteuert und gibt entweder Daten für ein Load zurück oder speichert Daten für eine Store-Anweisung. Die Daten kommen aus dem Datenspeicher und durchlaufen den Cache.

TODO: Überblick über die neuen Register geben.

TODO: matrix_operation_wait_i signal to sensitivity list

6 Auswertung und Evaluierung

7 Zusammenfassung und Ausblick

test

TODO: Beschreibe den Compiler hier:

Quellenverzeichnis

- [1] ALON AMID, et. a. Krste Asanovic A. Krste Asanovic: RISC-V "V"Vector Extension. <https://github.com/riscvarchive/riscv-v-spec/blob/master/v-spec.adoc>. Version: 2024
- [2] ARSALAN, Qizal: NEORV32 Hardware Accelerator (Arty A7) – Modified Core with CFU/CFS. https://github.com/QizalaA/neorv32_hardware_accelerator_arty_a7, 2025
- [3] ASANOVIC, Krste: RISC-V State of the Union. <https://riscv-europe.org/summit/2025/media/proceedings/2025-05-13-RISC-V-Summit-Europe-11h30-ASANOVIC%2086-slides.pdf>. Version: 2025
- [4] DAVID FLANAGAN, Adrian N.: X Toolkit Intrinsics Reference Manual: for X11 Release 4 and Release 5. The Associates, 1990
- [5] FALK, Sigurd: Ein übersichtliches Schema für die Matrizenmultiplikation. Berlin : Wiley-VCH, Zeitschrift für Angewandte Mathematik und Mechanik (ZAMM), 1951
- [6] HARRIS, Sarah L. ; HARRIS, David: Digital Design and RISC-V Computer Architecture Textbook. In: 2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE) (2021), 1-5. <https://api.semanticscholar.org/CorpusID:246872104>
- [7] INTERNATIONAL, RISC-V: Integrated Matrix Extension. <https://riscv.atlassian.net/wiki/spaces/IMEX/pages/46071925/Charter>. Version: 2024
- [8] KILLIAN, Earl A.: VME Design Considerations. <https://riscv.atlassian.net/wiki/download/attachments/663781400/DesignConsiderationsForVME.pdf?api=v2>. Version: 2025
- [9] LARABEL, Michael: Qualcomm's Xqci RISC-V Extension Now Deemed Non-Experimental For LLVM 22. <https://www.phoronix.com/news/Qualcomm-Xqci-LLVM-Stable>. Version: 2025
- [10] NOLTING, Stephan: neorv32. <https://github.com/stnolting/neorv32>. Version: 2026
- [11] NOLTING, Stephan: The NEORV32 RISC-V Processor - Data Sheet. <https://stnolting.github.io/neorv32/>. Version: 2026
- [12] TILO ARENS, FRANK HETTLICH, CHRISTIAN KARPfINGER, ULRICH KOCKEL-

KORN, KLAUS LICHTENEGGER, HELLMUTH STACHEL: Mathematik. SpringerSpektrum, Springer-Verlag, Berlin Heidelberg 2008, 2011, 2015, 2015

- [13] ZEE, Field G. ; GEIJN, Robert A.: BLIS: A Framework for Rapidly Instantiating BLAS Functionality. In: ACM Transactions on Mathematical Software 41 (2015), June, Nr. 3, 14:1–14:33. <https://doi.acm.org/10.1145/2764454>